Отчет по лабораторной работе 4

Реализованы пять вариантов умножения квадратных матриц double: ijk_seq, ijk_omp и ikj_seq, ikj_omp (parallel for по строкам i матрицы C), blocked_omp (блоки C распределяются между потоками через collapse(2), внутри блока порядок i-k-j). A и B заполняются LCG с фиксированным seed; каждый вариант сверяется с ijk_seq — во всех запусках check=OK (максимальная относительная ошибка 0, так как во всех вариантах c_{ij} суммируется по k в одном и том же порядке). Время — медиана 5 запусков после прогрева, C обнуляется перед каждым запуском вне замера; короткие умножения повторялись в цикле ≥ 0.1 с (omp_get_wtime() в MinGW тикает раз в 1 мс).

Система: Intel Core i5-9400F (6 ядер / 6 потоков, L1d 32 КБ, L2 256 КБ на ядро, L3 9 МБ), 2×16 ГБ DDR4-2666; Windows 10, GCC 16.2.0 (MSYS2), флаги -O3 -fopenmp. Запуск: OMP_NUM_THREADS=p ./matmul N block [варианты].

3.1. Влияние размера матрицы (6 потоков, блок 32)

N	ijk_seq, с	ijk_omp, с	ikj_seq, с	ikj_omp, с	blocked_omp, с
128	0.001750	0.000602	0.000625	0.000266	0.000277
256	0.025125	0.006875	0.005219	0.001672	0.002156
512	0.203000	0.040500	0.043750	0.008187	0.011312
1024	2.386000	0.406000	0.541000	0.100500	0.074000

3.2. Влияние числа потоков (N = 1024, блок 32; T_{ijk_seq} = 2.299 с, T_{ikj_seq} = 0.506 с)

Потоки	ijk_omp, с	S ijk	E ijk	ikj_omp, с	S ikj	E ikj	blocked_omp, с
1	2.034	1.13	1.13	0.502	1.01	1.01	0.4070
2	1.246	1.85	0.92	0.256	1.98	0.99	0.2120
4	0.568	4.05	1.01	0.114	4.44	1.11	0.1120
8	0.388	5.93	0.74	0.101	5.01	0.63	0.0895
16	0.341	6.74	0.42	0.086	5.88	0.37	0.0745

3.3. Влияние размера блока (N = 1024, 6 потоков; медиана трёх запусков программы)

Размер блока	Время blocked_omp, с	T_{ikj_seq} / $T_{blocked_omp}$
16	0.0980	5.16
32	0.0695	7.28
64	0.0990	5.11
128	0.0870	5.82

Вывод:

Порядок циклов важнее числа потоков: ikj_seq быстрее ijk_seq в 4.5 раза при N = 1024 (0.506 против 2.30 с) и в 2.8–4.8 раза на остальных N. В ijk внутренний цикл идёт по k, и B[k][j] читается по столбцу с шагом N·8 байт — из каждой кеш-линии используется одно число, а при N = 1024 шаг 8 КБ ещё и вызывает конфликтные промахи. В ikj внутренний цикл последовательно проходит строки B и C и использует каждую загруженную линию целиком (по -fopt-info-vec GCC векторизует оба варианта 16-байтными SSE2-векторами, так что разница именно в доступе к памяти). До 512 включительно все три матрицы (≤ 6 МБ) помещаются в L3, и самым быстрым оказывается ikj_omp, а блочность только добавляет накладных расходов; при N = 1024 (24 МБ > L3) лучше всех blocked_omp: блоки 32×32 (3·8 КБ = 24 КБ) живут в L1d и многократно переиспользуются. Поэтому оптимальный блок — 32: при 16 слишком короткие внутренние циклы и больше накладных расходов на обход блоков, при 64 и 128 три блока (96 КБ и 384 КБ) уже не помещаются в L1/L2. Ускорение растёт почти линейно до 4 потоков (ikj: 1.98 на 2, 4.44 на 4) и перестаёт заметно расти после 6 — у процессора 6 ядер без Hyper-Threading, 8 и 16 потоков — переподписка (эффективность падает до 0.6 и 0.4). Небольшой прирост на 16 потоках объясняется лучшей балансировкой: при static-разбиении на 6 потоков любое вмешательство ОС тормозит весь цикл, а мелкие порции сглаживают это. Сверхлинейное ускорение ijk_omp (S > p) — эффект кеша: потоки одновременно читают одни и те же столбцы B, и линия, загруженная в общий L3 одним потоком, используется остальными. Ограничения: 6 ядер, общий L3 и шина памяти, снижение турбо-частоты при загрузке всех ядер, фоновые процессы Windows (разброс между запусками программы до ~20%).